23CSE455 Design Patterns

Case Study Report

Team: B12

Case study: Design Pattern Identification and Evolution in a NestJS-Based Parking Management Application

Application: Smart Parking Reservation System

Date: 2026-10-10

Submission repository: 23CSE455_DP_CaseStudy

Students

Roll number	Student name	Batch
AM.SC.U4CSE23349	Samanthula Eswar	CSE
AM.SC.U4CSE23210	Bantupalli Harsha Vardhan	CSE
AM.SC.U4CSE23344	Rayudu Vamsi Krishna	CSE
AM.SC.U4CSE23350	Sammeta Pavan	CSE

Abstract

This report studies the design patterns present in the *Smart Parking Reservation System*, a NestJS/TypeORM backend with a Next.js/React frontend, inspected at baseline commit `940daee`. Part 1 reports a baseline candidate set of four confirmed pattern categories comprising seven recorded instances: the Strategy pattern (GoF, behavioural; 1 instance) and three architectural patterns — Repository (4 instances), Dependency Injection / Inversion of Control (1 instance) and Layered Architecture (1 instance). Part 2 implements three evolution changes on top of the baseline — dynamic slot allocation (a second Strategy instance), EV charging (introducing the **State** pattern) and real-time occupancy monitoring (introducing the **Observer** pattern) — each captured as a sequential diff and covered by passing tests. Part 3 compares all six confirmed pattern types — **Strategy**, **State**, **Observer**, **Repository**, **Dependency Injection** and **Layered Architecture** — across JavaScript, Python, Java and C++ on lines of code, cyclomatic complexity and process runtime.

Evidence policy. Every pattern claim in this report is supported by actual source evidence: verifiable inspected file paths, code excerpts, and source locations. LLM-generated assertions alone are insufficient and are not accepted as evidence for any pattern, pattern count, metric, or test result.

1 Application and Architecture

The case study inspects the *Smart Parking Reservation System*, a full-stack application comprising a NestJS backend and a Next.js frontend.

- Application: Smart Parking Reservation System (NestJS backend + Next.js frontend).
- Repository URL:
  `https://github.com/nuhu78/Smart-Parking-Reservation-System`
- Branch: `main`.
- Baseline commit: `940daeeab52d5a0b101ce0e76813bcfac88e01a1`
  (short `940daee`).

- Access date: 2026-10-10.
- Backend stack: NestJS 11, TypeScript, TypeORM and PostgreSQL.
- Frontend stack: Next.js, React and Zustand.
- Modules in scope: parking-slot management, vehicle management, reservation, payment, occupancy, EV charging, notifications.

Architecture overview. The backend is organised into NestJS feature modules — `auth`, `users`, `parking`, `slots`, `reservations`, `mail` and `seed` — arranged in a controller → service → repository layering and wired together through the Nest dependency-injection container. Persistence is mediated by TypeORM against PostgreSQL. The frontend consumes the REST API and holds authentication state in a Zustand store.

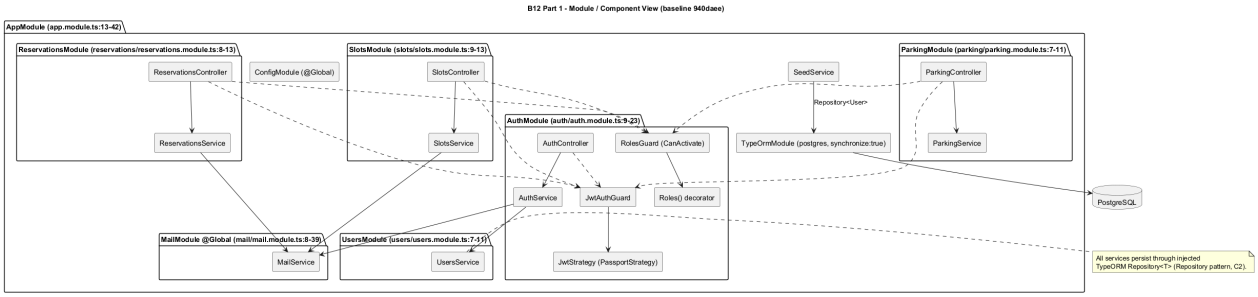


Figure 1: Application architecture at baseline commit `940daee`: module/component view of the `smart-parking-backend` wiring in `app.module.ts` and the feature modules (source: `Part1/diagrams/module_component.puml`).

Figure 1 shows the baseline module wiring; the note on `UsersService` records the Repository-pattern instances behind each service.

2 Part 1 Pattern Discovery

Part 1 was performed by direct inspection of the baseline source at commit `940daee`. Every pattern claim below cites a real, verified `file:line` location recorded in `Part1/evidence/pattern_evidence.md`. A pattern is marked *confirmed* only when the application source contains the pattern's participants, responsibilities and relationships; a framework class whose name merely resembles a pattern is not sufficient. Candidates that could not be established at application level are recorded as *uncertain* or *not found* in `Part1/evidence/rejected_and_uncertain.md`.

2.1 Totals at the Baseline

At baseline commit `940daee`, four pattern categories were confirmed, comprising **7 recorded instances** (from `data/pattern_counts.csv`); GoF and architectural patterns are distinguished explicitly.

Pattern category	Classification	Instances
Strategy	GoF (behavioural)	1
Repository	Architectural	4
Dependency Injection / IoC	Architectural	1
Layered Architecture	Architectural	1
Total		**7**

2.2 Pattern Inventory

The inventory is the machine-readable content of `data/patterns.csv` (7 data rows plus a header) and `data/pattern_counts.csv` (4 data rows plus a header). GoF patterns and architectural patterns are listed separately.

Pattern	Instance	Source location(s)
Strategy (GoF, behavioural)	JWT authentication strategy	`auth/strategies/jwt.strategy.ts:7-18`; `auth/guards/jwt-auth.guard.ts:1-5`; `slots/slots.controller.ts:22-23`
Repository (architectural)	User repository	`users/users.service.ts:9-12`
Repository (architectural)	ParkingArea repository	`parking/parking.service.ts:9-11`
Repository (architectural)	Slot repository	`slots/slots.service.ts:17-19`
Repository (architectural)	Reservation repository	`reservations/reservations.service.ts:15-16`
Dependency Injection / IoC (architectural)	Nest container wiring	`app.module.ts:13-42`; `users/users.module.ts:7-11`; `slots/slots.controller.ts:24-25`; `auth/auth.service.ts:18-22`
Layered Architecture (architectural)	Backend layering	`slots/slots.controller.ts:29-30`; `slots/slots.service.ts:29-49`; `parking/parking.controller.ts:27-28`; `parking/parking.service.ts:14-19`

Candidates recorded as *uncertain* (Singleton, Adapter, Template Method, Factory Method, Chain of Responsibility, Observer, Facade, Data Mapper) are **not** counted as confirmed. Rejected candidates (GoF Decorator, Builder, State, Proxy, Command, Iterator, Abstract Factory, MVC) are documented in `Part1/evidence/rejected_and_uncertain.md`.

2.3 One Record for Each Pattern Instance

Each record gives the pattern category, the instance, the participant roles and the verified source evidence. The full evidence register is in `Part1/evidence/pattern_evidence.md`.

Strategy (GoF, behavioural) — JWT authentication strategy. Concrete strategy: `JwtStrategy` (`auth/strategies/jwt.strategy.ts:7-18`), which implements the algorithm-specific `validate()` step (`auth/strategies/jwt.strategy.ts:22-30`). The strategy is selected polymorphically by name via `AuthGuard('jwt')` (`auth/guards/jwt-auth.guard.ts:5`) and used by guarded controllers such as `slots/slots.controller.ts:22-23`. The strategy interface and the context are supplied by `passport-jwt`/Passport; the application supplies the concrete strategy and its registration as a provider (`auth/auth.module.ts:22`). This attribution is stated explicitly rather than assumed from a class named `Strategy`.

Repository (architectural) — four instances. User (`users/users.service.ts:9-12`), ParkingArea (`parking/parking.service.ts:9-11`), Slot (`slots/slots.service.ts:17-19`) and Reservation (`reservations/reservations.service.ts:15-16`). In each case an application service depends on an injected typed `Repository<...>` (TypeORM) and mediates all persistence for that entity through it.

Dependency Injection / Inversion of Control (architectural) — one instance. Dependencies are declared in module `providers/exports` metadata and delivered through constructor injection — for example `app.module.ts:13-42`, `users/users.module.ts:7-11`, `slots/slots.controller.ts:24-25` and `auth/auth.service.ts:18-22`.

Layered Architecture (architectural) — one instance. One-directional call flow controller → service → repository, visible across the `slots`, `parking` and `reservations` modules (`slots/slots.controller.ts:29-30`; `slots/slots.service.ts:29-49`; `parking/parking.controller.ts:27-28`; `parking/parking.service.ts:14-19`).

3 Part 2 Evolution and Refactoring

Three evolution changes were implemented as additive overlays on the baseline backend. The original application source was *not* modified in place; each change exists as a set of changed and new source files under `Part2/code/evolved/smart-parking-backend/src/` and a unified diff under `Part2/changes/ChangeN/diff/`. Diffs are sequential: each applies on top of the previous change's snapshot. Every change was compiled and its tests were run with Jest 30.2.0 on Node 24.18.0 (results summarised in `test_result/part2_summary.md`; raw logs per change).

3.1 Change 1 — Dynamic Slot Allocation (Strategy)

Requirement. Replace the hard-coded "first available slot" choice with a pluggable allocation policy (first available, nearest to entrance, EV preferred) selectable per request. **Pattern.** The existing baseline Strategy provided by Passport/JWT covers authentication only; this change introduces an application-level Strategy of its own — a `SlotAllocationStrategy` interface with three concrete strategies and a `SlotAllocationContext` that dispatches to the selected policy (default `first-available`, unknown names falling back to it). An additive nullable `distanceToEntrance` column was added to the slot entity to support the nearest-to-entrance policy; existing slot methods are unchanged. **Diff.** `Change1/diff/change1.diff` — 10 files, +249/ − 1. **Test.** `Change1/test_result/jest_change1.txt` — 1 suite, 8/8 pass.

Figure 2 shows the participant structure of the new Strategy.

3.2 Change 2 — EV Charging (State)

Requirement. Model an EV charging session whose behaviour changes with its lifecycle state.

Pattern.

This introduces the **State** pattern (recorded as rejected/not-found at baseline): a `ChargingState` interface with `Idle`, `PluggedIn`, `Charging` and `Completed` concrete states held by a `ChargingSession` context. Legal transitions are `idle` → `plugged-in` → `charging` → `completed` → `idle`, plus an unplug during charging; illegal operations raise `BadRequestException`. A `ChargingService` / `ChargingController` expose the session over REST.

Diff. `Change2/diff/change2.diff` — 8 files, +302/ − 1 (`app.module.ts` adds `ChargingModule`).

Test. `Change2/test_result/jest_change2.txt` — 1 suite, 8/8 pass.

Figure 3 shows the State-pattern participants and the legal transitions exercised by the session.

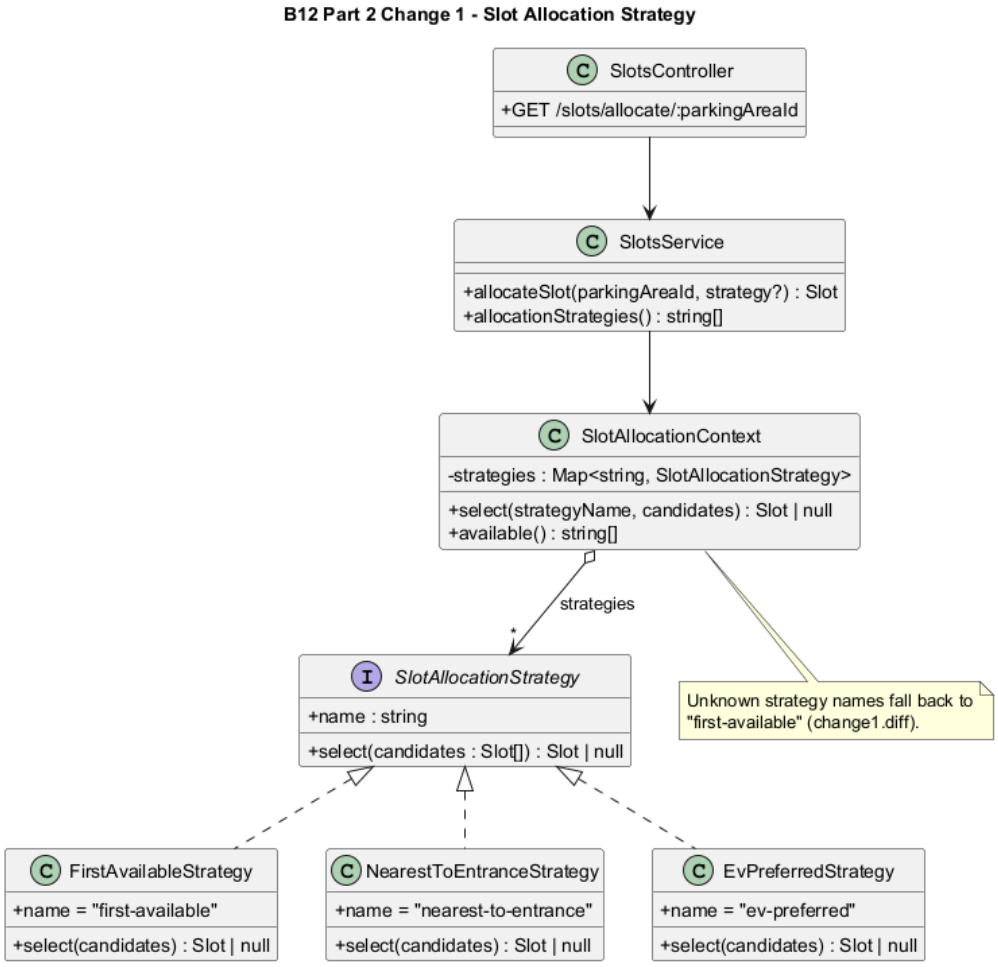


Figure 2: Change 1 (Strategy): `SlotAllocationStrategy` interface, its three concrete strategies (`first-available`, `nearest-to-entrance`, `ev-preferred`) and the `SlotAllocationContext` dispatch (source: `Part2/diagrams/change1_strategy.puml`).

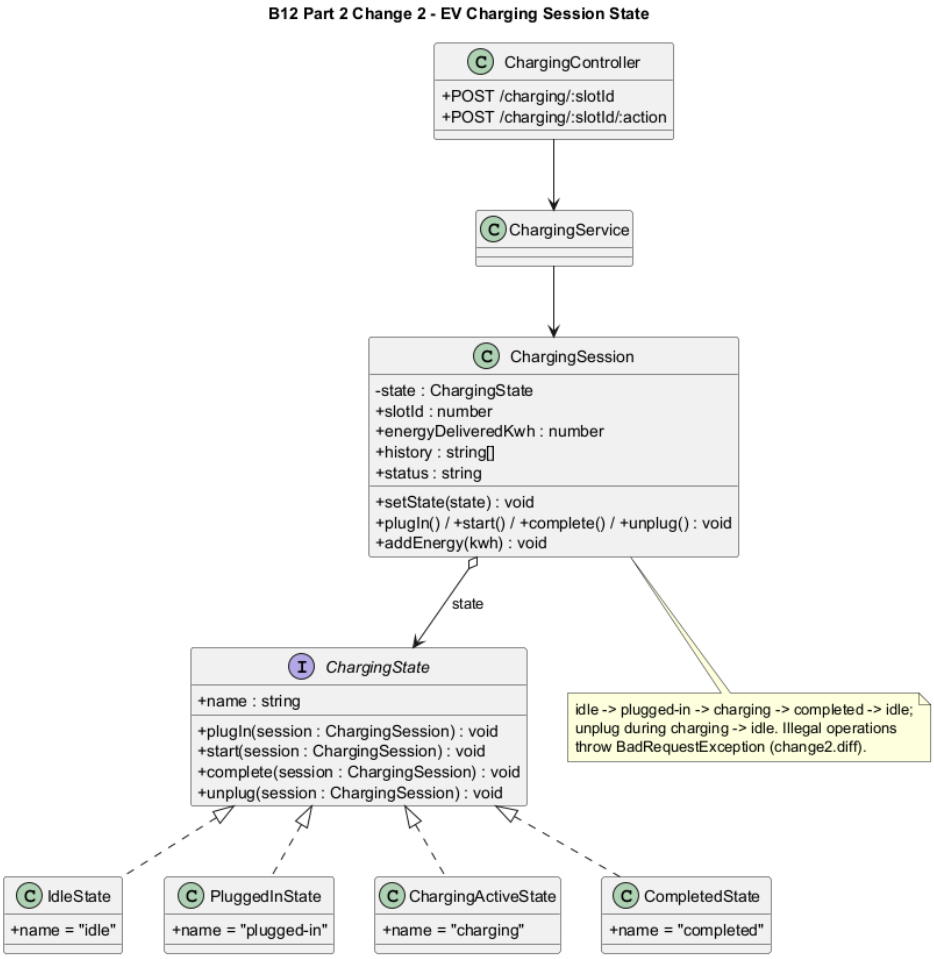


Figure 3: Change 2 (State): `ChargingState` interface, the four concrete states and the `ChargingSession` context (source: `Part2/diagrams/change2_state.puml`).

3.3 Change 3 — Real-Time Occupancy Monitoring (Observer)

Requirement. Notify interested parties whenever a slot's occupancy changes. **Pattern.** This introduces the **Observer** pattern: `OccupancyMonitor` is the subject and `OccupancyObserver` the observer interface, with a `LoggingObserver` (appends an event log) and an `AvailabilityObserver` (tracks occupied slots and a count). Observers can be attached and detached at runtime, and are wired in the module. A controller accepts occupancy updates. **Diff.** `Change3/diff/change3.diff` — 8 files, +177/ − 1 (`app.module.ts` adds `OccupancyModule`). **Test.** `Change3/test_result/jest_change3.txt` — 1 suite, 5/5 pass.

Figure 4 shows the Observer-pattern subject, observers and the occupancy event flow.

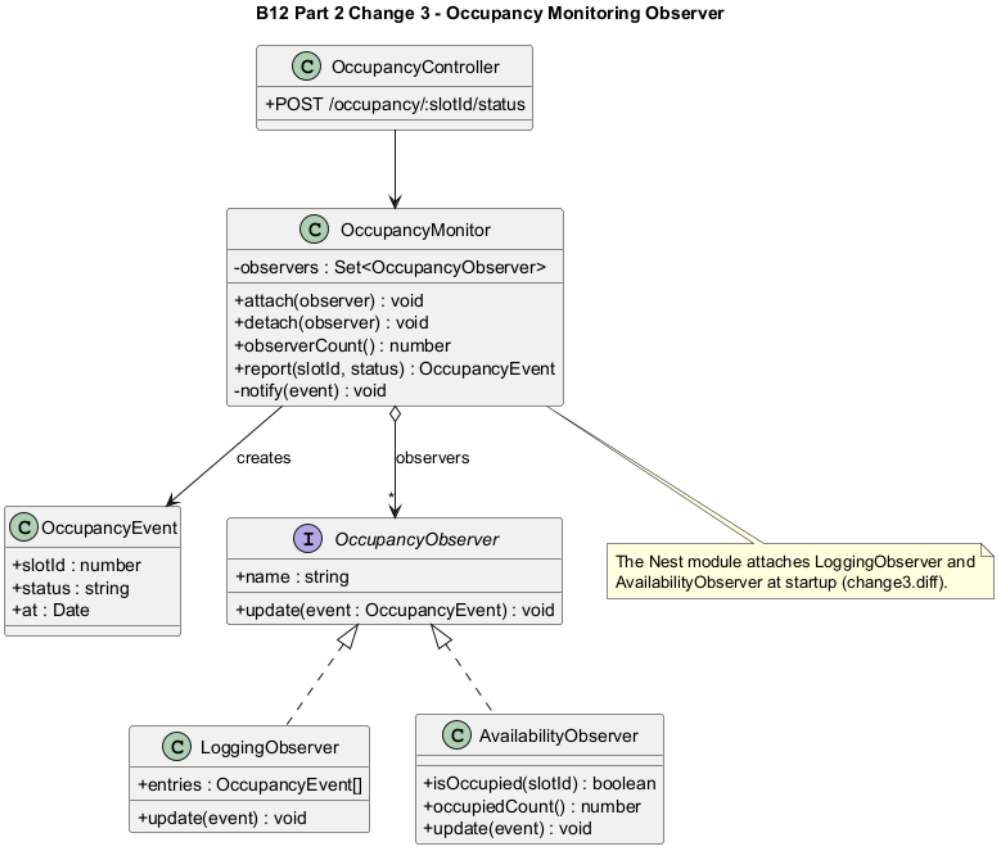


Figure 4: Change 3 (Observer): `OccupancyMonitor` subject, the `OccupancyObserver` interface with `LoggingObserver` and `AvailabilityObserver` (source: `Part2/diagrams/change3_observer.puml`).

3.4 Updated Pattern Counts

The table below compares the baseline counts with the final post-change counts (also recorded incrementally in `data/pattern_counts.csv`). Change 1 adds a second Strategy instance (slot allocation) alongside the baseline JWT strategy; Change 2 introduces State; Change 3 introduces Observer. Repository, DI/IoC and Layered Architecture remain as at baseline and are reused by the new code.

Pattern category	Baseline	After Part 2
Strategy	1	2
State	0	1
Observer	0	1
Repository	4	4
Dependency Injection / IoC	1	1
Layered Architecture	1	1
Total	**7**	**10**

Regression status. Across all three changes the full backend suite reports 22 passed, 3 failed out of 25. The 21 new Part 2 tests all pass; the 3 failures are pre-existing baseline scaffold defects in `auth.controller.spec.ts`, `auth.service.spec.ts` and `mail.service.spec.ts` (Nest cannot resolve their dependencies), which fail identically on the untouched baseline and are unrelated to Part 2. Evidence: `test_result/part2_jest_full.txt` and `test_result/baseline_preexisting_failures_raw.txt`.

4 Part 3 Cross-language Comparison

The six confirmed pattern types central to this case study — **Strategy** (slot allocation), **State** (EV charging session), **Observer** (slot occupancy notification), **Repository** (user account persistence), **Dependency Injection** (notification channel injection) and **Layered Architecture** (reservation request handling) — were each implemented from scratch in JavaScript, Python, Java and C++, every implementation with a self-contained test. All 24 suites pass (Strategy 5/5, State 13/13, Observer 8/8, Repository 8/8, Dependency Injection 6/6 and Layered Architecture 8/8 in every language). The comparison records lines of code (production snippet only, excluding blanks, comments and tests), maximum function cyclomatic complexity, and median whole-process wall time (one warm-up discarded, five runs, high-resolution timer; compile time excluded). Rules and per-run logs are in `Part3/evidence/` and the snippets under `Part3/<Pattern>/<lang>`. This scope — one representative instance of *every* confirmed pattern type — follows the instruction "for each detected pattern type, implement its small problem in all four languages"; the decision is recorded in `Part3/Part3_scope_decision_note.md`.

Pattern	Language	LOC	Max CC	Median ms
Strategy	JavaScript	75	5	80.87
Strategy	Python	49	6	112.12
Strategy	Java	77	7	185.65
Strategy	C++	70	8	18.98
State	JavaScript	94	1	73.15
State	Python	57	1	67.42
State	Java	83	1	173.18
State	C++	97	2	20.83
Observer	JavaScript	56	3	75.32
Observer	Python	33	3	65.18
Observer	Java	44	3	159.11
Observer	C++	44	3	15.94
Repository	JavaScript	58	2	78.93
Repository	Python	43	1	104.62
Repository	Java	56	1	143.41
Repository	C++	46	2	16.19
Dependency Injection	JavaScript	43	1	80.49
Dependency Injection	Python	25	1	45.41
Dependency Injection	Java	36	1	141.08
Dependency Injection	C++	34	1	17.05
Layered Architecture	JavaScript	51	2	74.70
Layered Architecture	Python	34	3	89.70
Layered Architecture	Java	53	3	148.05
Layered Architecture	C++	48	3	15.81

Python produces the shortest snippets; Java and C++ are the most verbose; the statically linked C++ binaries are roughly an order of magnitude faster, while the timed Java and Python figures are dominated by VM/interpreter startup. Cyclomatic complexity is low throughout: the State, Dependency Injection and (most) Repository snippets are branch-free in the garbage-collected languages (CC 1) because behaviour is delegated to polymorphic or injected collaborators. Strategy has the highest complexity — C++ reaches 8 through the null guards its pointer-returning `choose()` requires — while Observer carries a subscriber loop and a detach guard (CC 3) in every language. All values are taken directly from `data/cross_language.csv`.

5 LLM Records and Human Verification

One LLM artifact is recorded for this case study: a shared ChatGPT conversation about the assignment, kept as a research artifact only. No LLM was used to produce the Part 2 or Part 3 code, tests or metrics, and no new prompt was issued for Part 2 (see `Part2/prompts/Part2_prompt.txt`). The full record is in `data/llm.csv`; the reference index is `LLM_RESEARCH_REFERENCES.md`.

Part	Model / date	Prompt / verification
All (research artifact)	ChatGPT shared conversation (exact model and date not shown)	Prompt: share page. Verification: **not independently verified**; treated as a research artifact only, never as evidence for any pattern, count, metric or test result. Index: `LLM_RESEARCH_REFERENCES.md`.

> LLM outputs are recorded as artifacts only. Every claim derived from an LLM response must be independently verified against the actual source code before it may appear anywhere in this report. LLM assertions alone are insufficient.

6 Findings, Contributions and References

Findings. (i) At baseline the application confirms four pattern categories and seven instances (Strategy, four Repository instances, DI/IoC and Layered Architecture); payment, EV charging and occupancy monitoring are absent as modules. (ii) The three Part 2 changes each fit an existing architectural style: Change 1 adds an application Strategy alongside the Passport/JWT Strategy, Change 2 introduces State, and Change 3 introduces Observer — the last two having been recorded as rejected/not-found at baseline. (iii) All 21 new tests pass; the only suite failures are three pre-existing baseline defects unrelated to the changes. (iv) In the cross-language study all six confirmed pattern types were implemented in the four languages; the State, Dependency Injection and most Repository variants are branch-free in the garbage-collected languages, Python is the most concise, Java the most verbose, and the native C++ binary is roughly an order of magnitude faster than the VM/interpreter languages when timing includes runtime startup.

Contributions. Real, runnable artifacts for Parts 2 and 3; per-change requirement notes, sequential diffs and raw test logs; a reproducible cross-language benchmark with raw evidence; and machine-readable datasets (`patterns.csv`, `pattern_counts.csv`, `cross_language.csv`, `llm.csv`).

References.

- `Smart Parking Reservation System` (baseline source; commit `940daee`): `https://github.com/nuhu78/Smart-Parking-Reservation-System`
- Assignment repository (course): `https://github.com/aswathymohan-amrita/23CSE455_DP_CaseStudy`
- Recorded LLM research artifact (ChatGPT shared conversation): `https://chatgpt.com/share/6aca22a9-2844-83ec-8126-e6f81ef8e85e`

Formal bibliographic citations (for example the GoF book) are not yet included in this section.

7 Limitations

- This is a baseline source-code study. It does not claim that any missing feature has been, or will be, implemented.
- At the **baseline** the following were **not found** in the inspected source: payment, EV charging, occupancy monitoring as a module, and a dedicated vehicle-management module. Vehicle-related data exists only as fields on the `User` and `Reservation` entities. EV charging and occupancy monitoring were *added* by the Part 2 overlay and were not present in the original application.
- Framework- and library-provided behaviour (for example NestJS providers, TypeORM repositories, Passport strategies and mailer adapters) must not automatically be counted as an application-level GoF pattern.
- The first Strategy instance relies on the Passport/JWT strategy integration supplied by the application (`JwtStrategy`) within the context provided by Passport; the second (slot allocation) is entirely application-level and was introduced in Part 2. Both attributions are valid only within the evidence and scope stated in this report.
- Pattern counts follow the counting rule documented in the Part 1 report and cover both the baseline commit and the Part 2 overlay snapshots; they are not performance or quality metrics.

- The baseline backend has no committed `node_modules` and its `package-lock.json` is out of sync, so `npm ci` fails; the Part 2 tests were therefore run with `npm install` inside a disposable copy of the backend, and the baseline itself was never modified.
- The Part 2 diffs were produced from sequential snapshots of only the added/changed files; they are not full-repository diffs.

8 Team Folder and Datasets

Team folder: `B12/`

- `data/patterns.csv` — one row per baseline pattern instance (7 data rows plus a header)
- `data/patterns_evolution.csv` — per-version pattern evolution records (34 data rows plus a header)
- `data/pattern_counts.csv` — per-version pattern counts, with per-pattern and total rows (27 data rows plus a header)
- `data/cross_language.csv` — cross-language comparison records (24 data rows plus a header)
- `data/llm.csv` — LLM prompt/input/output/verification records (1 data row plus a header)
- `Part1/`, `Part2/`, `Part3/` — per-part reports, code, evidence, diagrams, prompts and LLM artifacts
- `test_result/` — raw Jest logs and the Part 2 summary (`part2_summary.md`, `part2_jest_full.txt`, `baseline_preexisting_failures_raw.txt`)

`patterns.csv` (7 records) and `patterns_evolution.csv` (7 baseline records plus 27 post-change records) hold the Part 1 and Part 2 results; `pattern_counts.csv` (6 baseline records plus 21 post-change records, per-pattern and total rows) summarises the counts per version. `cross_language.csv` holds the 24 Part 3 comparison records and `llm.csv` the single honest LLM record.